

P 与 NP 问题

Stephen Cook (University of Toronto)

1. 问题的表述

P 与 NP 问题旨在确定是否每个被某个非确定性算法可在多项式时间内接受的语言, 也可被某个(确定性)算法在多项式时间内接受. 要精确地详述这一问题必须先有一计算机的形式模型. 在可计算性理论中标准的计算机模型是 Turing 机, 这是 1936 年由 Alan Turing 引进的模型 [Tur 37]. 虽然这一模型在真实的计算机被造出来之前就被引进了, 但为定义可计算函数概念起见, 它仍然被看作是适宜的计算机模型.

非形式地讲, 类 P 中的问题恰好是那些可以由某算法用其输入的长度的某个取定的多项式界限的步数可解出的判定问题. Turing 并不考虑他的机器的效率, 而是关注于它们是否可以在给定的充分多的时间内模拟任何的算法. 但是可以看出, 一般地, Turing 机可以最多用计算时间的平方或立方那么多的时间来模拟更有效的计算机模型(例如装有多条带子的或装有一无穷随机存取存储装置的机器). 因此 P 是一鲁棒类(robust class), 并且在大量的计算机模型类上有等价的定义. 在这里我们按标准的惯例, 用 Turing 机定义类 P.

形式地讲, 类 P 的元素是语言, 令 Σ 是一有穷字母表(即一有穷非空集), 且令 Σ^* 是 Σ 上有穷串的集. 那么 Σ 上的一语言是 Σ^* 的一子集. 每个 Turing 机 M 有一相伴的输入字母表 Σ . 对每个 Σ^* 中的串 w 有一个与 M 和其输入 w 相伴的计算. (Turing 机和计算的概念会在附录中正式定义). 我们称 M 接受 w , 若这计算终止于可接受状态. 注意, 如果这计算终止于拒绝状态或者计算无法终止, 那么 M 就不能接受 w . 被 M 接受的语言, 记为 $L(M)$, 有相伴的字母表 Σ , 定义为

$$L(M) = \{w \in \Sigma^* | M \text{ 接受 } w\}.$$

我们以 $t_M(w)$ 表示当输入是 w 时 M 的计算的步数(见附录). 若此计算一直不停止, 则 $t_M(w) = \infty$. 对 $n \in \mathbb{N}$, 我们以 $T_M(n)$ 表示 M 的在最坏情形下的运行时间, 亦即

$$T_M(n) = \max\{t_M(w) | w \in \Sigma^n\}$$

其中 Σ^n 是 Σ 上一切长度为 n 的串之集. 我们称 M 在多项式时间内运行, 如果有一 k 使得对一切 n , $T_M(n) \leq n^k + k$. 现在我们定义语言的类 P 为(即“确定性多项式时间类”)

$$P = \{L | \text{有某一个在多项式时间内运行的图灵机 } M \text{ 使得 } L = L(M)\}$$

记号 NP 表示“非确定性多项式时间类”, 因为最初 NP 是用非确定性机器(即, 由一给定的配置, 机器可能有多于一种的可能的处置)来定义的. 但是现在通常使用

一校验关系的等价定义, 这校验关系只不过是对于某有穷字母表 Σ 和 Σ_1 的一个二元关系 $R \subseteq \Sigma^* \times \Sigma_1^*$ 对每个这样的关系 R 我们对应地定义一个: $\Sigma \cup \Sigma_1 \cup \{\#\}$ 上的语言 $L_R, L_R = \{w\#y | R(w, y)\}$. 其中符号 $\#$ 不在 Σ 之中, 我们称 R 是一多项式时间关系, 当且仅当 $L_R \in P$.

现在我们用如下条件定义语言的类 NP: Σ 上的语言 L 是属于 NP 的当且仅当有 $k \in \mathbb{N}$ 和一多项式时间的校验关系 R , 使得对一切 $w \in \Sigma^*$ 有

$$w \in L \iff \exists y (|y| \leq |w|^k \text{ 且 } R(w, y))$$

其中 $|w|$ 和 $|y|$ 分别表示 w 和 y 的长度.

问题陈述: $P=NP$ 吗?

显然可证 $P \subseteq NP$, 因为对每个 Σ 上语言 L , 若 L 属于 P 那么我们对一切 $w, y \in \Sigma^*$ 用

$$R(w, y) \iff w \in L$$

来定义多项式时间校验关系 $R \subseteq \Sigma^* \cup \Sigma^*$.

这里我们用十进位记号对自然数进行编码来给出两个简单的例子: 完全平方数之集属于 P 且合数之集属于 NP (尚不知是否属于 P). 后者的伴随多项式时间校验关系 R 可定义如下:

$$R(\bar{a}, \bar{b}) \iff 1 < b < a \text{ 且 } b|a \quad (1)$$

一般情况下自然数 c 的拾进位记号记为 $\bar{c}$.

2. 历史和重要性

P 与 NP 问题的重要性, 部分地来源于 NP 完全性的成功的理论, 部分地来源于密码学. 对诸如可满足性这样的 NP 完全性问题的任何一个真正有效的算法, 一方面能对工业中许多实用的计算问题提供有用的算法, 另一方面也会破坏广泛用于互联网的金融和其它一些财务往来的安全性. 这里我们简要地给出有关 NP 完全性的历史并且涉及密码学理论.

NP 完全性理论植根于可计算理论之上, 而可计算性理论则起源于二十世纪 30 年代 Turing, Church, Gödel 和其他一些人的工作之上. P 与 NP 类的可计算性的前导分别是可判定类和 c.e. (可计算枚举) 的语言. 我们说一语言 L 是 c.e. 的 (或者称为 半可判定) 当且仅当 $L = L(M)$, 其中 M 为某 Turing 机. 我们说 L 是可判定的当且仅当有对一切输入串都停机的 Turing 机 M 使 $L = L(M)$. 关于 c.e. 的一个等价定义可以显示出 c.e. 和 NP 的相似性, 即 L 是 c.e. 的当且仅当有一满足 $L = \{x | \exists y R(x, y)\}$ 的可计算 "校验关系" $R(x, y)$.

以记号 $\langle M \rangle$ 表示描绘 Turing 机 M 的串, 我们定义停机问题 HP 为:

$$HP = \{\langle M \rangle | M \text{ 是一个对输入 } \langle M \rangle \text{ 停机的 Turing 机}\}$$

Turing 用一简单的对角线论证证明了 HP 是不可判定的. 另外不难证明 HP 是 c.e. 的.

可计算性理论中带有核心重要性的是可归约性概念, 有关这概念 Turing 大致地定义如下: 一语言 L_1 是 Turing 可归约到一语言 L_2 的, 当且仅当有一带信息源的 Turing 机 M 使 M 接受 L_1 , 其中 M 被允许提出形式为 “ $x \in L_2$?” 这样的归属询问, 而这种询问可被有关 L_2 的信息源正确地回答出. 后来又引进了更具限定性的多一可归约性 (many-one reducibility, 记作 $\leq_m$) 的概念, 其定义如下.

定义 1 设 L_i 是 Σ_i 上的语言, $i = 1, 2$, 则 $L \leq_m L_2$ 当且仅当有一 (全) 可计算函数 $f: \Sigma^* \rightarrow \Sigma_2^*$ 使得对一切 $x \in \Sigma_1^*, x \in L_1 \iff f(x) \in L_2$.

容易看出若 $L_1 \leq_m L_2$ 且 L_2 是可判定的, 那么 L_1 是可判定的. 这事实给了我们一个证明不可判定性的重要工具; 例如若 $HP \leq_m L$ 则 L 是不可判定的.

NP 完全性的概念是基于可计算理论的如下概念的:

定义 2 一语言 L 是 c.e. 完全的, 当且仅当 L 是 c.e. 的并且对一切 c.e. 的语言 L' 有 $L' \leq_m L$.

容易证明 HP 是 c.e. 完全的. 现在知道大多数最 “自然的” 不可判定的 c.e. 语言实际上是 c.e. 完全的. 由于 $\leq_m$ 是可传递的, 要证明某 c.e. 语言 L 是 c.e. 完全的, 只要证明 $HP \leq_m L$.

多项式时间计算的概念是 20 世纪 60 年代由 Cobham [Cob64] 和 Edmonds [Emd65] 作为计算复杂性理论的早期发展而引进的 (尽管早在 1953 年 von Neumann[vN] 就把多项式时间算法和指数时间算法加以区别). Edmonds 把多项式时间算法称为 “好算法”, 并且把它们与易处理算法相联系.

现在在复杂性理论中把多项式时间和可行的等同起来是通行的看法, 这里我们离开主题就这点再说两句, 当然对小的输入的每个多项式时间算法可以被可行地执行这点并不是千真万确的; 例如一个需要 100^n 步的计算机程序甚至对 $n = 10$ 这样的输入也决不能被执行. 这里给出一个更能辩护的陈述.

可行性论题 一个自然问题有一可行算法当且仅当它有一多项式时间算法.

那些很自然地既是可行的又是多项式时间算法的问题是很多的: 整数算术, 线性代数, 网络流, 线性规划, 许多图问题 (连通性, 最短路径, 最小生成树, 二图匹配) 等等. 另一方面 Robertson 和 Seymour 的深刻结果 [RS83] 提供了对此论题的反例的一潜在的来源: 他们证明了图的每个子式封闭族 (minor-closed family) 可在多项式时间 (实际上是 $O(n^3)$) 识别, 但是他们给出的方法所提供的算法有如此大的常数, 这些常数不是可行的. 但是每个潜在的反例可藉找到它的一个可行的算法而消除. 例如对平面图类已经有了识别的算法, 但是当前对可嵌入 $\mathbb{R}^3$ 不具有二个相连接的圈的图类尚不知有可行的算法. (这两个例子都是子式封闭族). 当然在上述论题中的 “自然的” 和 “可行的” 字样应予以解释; 一般说来我们并不认为带有参数的类是自然的类, 例如可嵌入在一亏格 $k, k > 1$ 的表面的图集. 对可行性论题更多的讨论见 [Coo90].

量子计算机对这论题的 “仅当” 方向的反例提供了潜在的来源. 这种计算机模型溶入了量子机器的状态的迭加的想法, 并且允许 Turing 机上某些计算中潜在的指数飞速

增长. 例如 Shor[Sho97] 曾证明了某些量子计算机的算法可以在多项式时间把整数因子分解, 但是对 Turing 机尚不知道有可在多项式时间内实现整数因子分解的算法. 但是迄今为止物理学家尚不能造出可处理多于半打的位数的量子计算机, 所以目前它们对可行性论题的威胁还只是臆测的.

回到复杂性理论历史方面的论述. 1971 年本作者在文 [Coo71] 中引进了 NP 完全性的概念作为 c.e. 完全性的多项式时间的类比, 除了所用的归约是 Turing 可归约性的多项式时间的类似概念, 而不是多一可归约性的多项式时间的类似概念. 文 [Coo71] 中的主要结果包括可满足性和 (下面定义的) 3-SAT 以及子图同构问题在内的一些自然的问题都是 NP 完全的. 一年以后 Karp[Kar72] 用这些完全性结果证明了 20 个其他的自然的问题是 NP 完全的, 因此有说服力地证明了这主题的重要性. Karp 也引进了现在的标准记号 P 和 NP 并且用多一可归约性的多项式时间的类似物重新定义 NP 完全性, 这定义已成为标准的定义, 同一时候 Levin[Lev73] 独立于 Cook 和 Karp 定义了类似于 NP 完全性问题的“通用搜索问题”并给出包括可满足性在内的 6 个例子.

有关 NP 完全性的标准定义接近于上述的定义 1 和 2.

定义 3 设 L_i 是 $\Sigma_i (i=1, 2)$ 上的语言, 那么 $L_1 \leq_P L_2$ (L_1 是 P 可归约到 L_2) 当且仅当有一多项式时间可计算函数 $f: \Sigma_1^* \rightarrow \Sigma_2^*$, 使得对一切 $x \in \Sigma_1^*, x \in L_1 \iff f(x) \in L_2$.

定义 4 一语言 L 是 NP 完全的, 当且仅当 L 是 NP, 对一切 NP 中的语言 $L', L' \leq_P L$. 容易证明以下的命题: (b) 部分用 $\leq_P$ 的传递性, (c) 部分可由 (a) 部分推出.

命题 1 (a) 若 $L_1 \leq_P L_2$ 且 $L_2 \in P$ 则 $L_1 \in P$.

(b) 若 L_1 是 NP 完全的, $L_2 \in \text{NP}$, 且 $L_1 \leq_P L_2$ 则 L_2 是 NP 完全的.

(c) 若 L 是 NP 完全的且 $L \in P$, 则 $P = \text{NP}$.

注意 (a) 和 (b) 部分在可计算理论中有密切相似的命题, (c) 部分的相似命题可以简单地陈述为: 若 L 是 c.e. 完全的, 那么 L 是不可判定的. (b) 部分是证明新的问题为 NP 完全时要用的基本方法, 而 (c) 部分说明为什么对一 NP 完全性问题寻找一多项式时间算法也许只会浪费时间.

通常 NP 的元素都以一种判定问题来表述, 相应的语言就是在标准的编码方法下编制这个判定问题的肯定例证的串组成的集. 因此可满足性问题是: 给定一命题公式 F , 判定 F 是否是可满足的. 要证明这是 NP 问题, 我们定义多项式时间校验关系 $R(x, y)$. $R(x, y)$ 成立当且仅当 x 编码一命题公式 F , 且 y 编码使 F 为真的 F 变元的一真假值赋值. 在 [Coo71] 中证明了这是 NP 完全问题, 其方法主要是对每个识别某 NP 语言 L 的校验关系 $R(x, y)$ 的多项式时间 Turing 机 M , 有一多项式时间算法, 它取一串 x 为输入且产生一命题公式 F_x (描述 M 对输入 (x, y) 的计算并带有表示未知的串 y 的变元) 使得 F_x 可满足, 当且仅当有满足 $|y| \leq |x|^k$ 的某 y 使 M 接受输入 (x, y) .

可满足性的一个特别重要的例子是 3-SAT, 它也在文 [Coo71] 中被证明为 NP-完全的, 3-SAT 的例子只限于形如合取范式的公式, 在其中每个子句有三个字母, 例如公式

$$(P \vee Q \vee R) \wedge (\bar{P} \vee Q \vee \bar{R}) \wedge (P \vee \bar{Q} \vee S) \wedge (\bar{P} \vee \bar{R} \vee \bar{S}) \quad (2)$$

是 3-SAT 的可满足例句, 因为规定为 $\tau(P) = \tau(Q) = \text{真}$ 并且 $\tau(R) = \tau(S) = \text{假}$ 的赋值 τ 就可使此式取真值.

已经确认了几百个 NP 完全的问题, 包括子集和问题 (给定一表为十进位数的正整数集和一个目标 T , 是否有一子集其和为 T), 许多图问题 (给定一图 G , G 是否有 Hamilton 圈? 是否有一由其一半顶点组成的团 (clique)? 是否可用三种颜色对 G 的顶点着色使得相邻的顶点着不同的色?). 这些问题导致工业中许多重要的进度安排和路线安排问题. 书 [GJ79] 提供了这个主题的精彩的参考材料, 其附录中列举了 300 个 NP 问题.

与每个 NP 中的判定问题相伴随的有一搜索问题, 这就是给定串 x 要找满足此问题校验关系 $R(x, y)$ 的一个串 y (或者确定 x 是问题的一个否定例证). 这样的 y 称为 x 的一个凭证. 当这一问题是 NP 完全时, 容易看出这个搜索问题可以有效地归约为相应的判定问题, 使得若 $P=NP$ 那么所有的相伴的搜索问题都有多项式时间算法, 例如对可满足性的判定问题的算法可以用来寻求一可满足公式 F 的满足性赋值, 方法是逐次地对 F 中的每个变元 P 赋以真值或假值, 看看哪一种赋值 τ 仍能保持 F 的可满足性.

有很有趣的 NP 问题例子, 我们不知它是否属于 P 或 NP 完全. 一个例子是第一节中讲的有校验关系 (1) 的例子. 有猜测认为, 不存在多项式时间归约把这搜索问题 (整数因子分解) 化为判定问题, 特别地 Miller[Mil76] 证明如何在多项式时间内判定一给定数是复合数, 这里假定了扩充的 Riemann 假设, 但是多项式时间的整数因子分解算法不像是存在的. 实际上一有效的因子分解算法会破解 RSA 公钥密码体制 [ARS78], 而这是一般用在互联网 (internet) 上传递金融信息所用的.

NP 完全的语言的补集被认为不在 NP 中; 否则每个 NP 语言的补集必在 NP 之中. 但是复合数集的补集 (本质上是素数集) 可用 Pratt 提出的有趣方法证明是在 NP 中 [Pra75], 因此不像是 NP 完全的.

这里存在着其复杂性等价于整数因子分解问题的 NP 判定问题, 即

$$L_{fact} = \{\langle a, b \rangle \mid \exists d (1 < d < a \text{ 且 } d|b)\}$$

给定整数 $b > 1$, 可用二元搜索藉对 L_{fact} 提问 $\log_2 b$ 次而找到 b 的最小素因子. 使用 Pratt 定理易证明 L_{fact} 的补集也在 NP 之中, 表明 $\langle a, b \rangle$ 是 L_{fact} 的一否定例证的凭证可以是 b 的完全素因子分解, 加上 Pratt 的凭证可以证明每个素因子确为素数, 因此普遍认为 L_{fact} 不像是 NP 完全的.

在现代密码学中, 计算复杂性理论扮演了重要角色 [Gol99]. 包括大多数金融往来在内的互联网安全性, 依赖于复杂因子分解或破解 DES (数据密码标准) 的难度. 若 $P=NP$, 这些假设不成立. 特别地, 能在 n^2 步内解出 3-SAT 的算法可用来在几分钟内完成对长达 200 位的数的素因子分解.

在 [Sma98] 文中, Smale 把 P 与 NP 问题列为 21 世纪中要解决的数学问题中的第三个. 但是 Smale 感兴趣的不仅是这问题的经典陈述, 还包括用复数域表达的陈述. 不过在这里 Turing 机要换成可以对任意复数进行严格算术运算和复数域上零的检验的机器模型. P 与 NP 问题就被换成一与 Hilbert 零点命题有关的问题: 是否有多项式时间算

法, 当被给定了 k 个多变元多项式集后, 判定它们是否有公共零点. 有关这方面的复杂性理论的发展见 [BCSS98], 其中包括 Mandelbrot 集是不可判定的在内的迷人的结果.

3. 猜测, 以及证明它的一些尝试

大多数复杂性理论工作者相信 $P \neq NP$. 也许这可通过对 $P=NP$ 和 $P \neq NP$ 这两种可能性的考虑来说明.

首先假定 $P=NP$ 并且考虑人们如何来证明它. 明显的方法是对 3-SAT 或其他 1000 个左右已知是 NP 完全的问题之一给出一多项式时间算法, 实际上许多错误的证明都是用这种形式表达的. 我们有为设计多项式时间算法的一个标准的可利用的工具箱 [CLR90], 其中包括渴望方法, 动态规划, 归约到线性规划等等, 它们是大学本科生计算机科学必修课中典型的有关算法的课程中的各种主题. 由于它们在工业中的重要性, 在过去三十年里大量的程序员和工程师企图寻找对 NP 完全性问题的有效算法, 但都未成功. 也有破解密码模式的相似的强烈的动机, 这些密码模式为了它们的安全性而假定了 $P \neq NP$.

当然象早先谈到可行性论题时提到的 Robertson/Seymour 的证明那样, 有可能用某种非构造性方法来证明 $P=NP$. 但这种证明不会给出标准 NP 完全性问题的任何可行算法. 无论如何, 到目前为止解 3-SAT 算法的最好的已证明的上界大约是 1.5^n , 其中 n 是输入公式中变元的个数.

另一方面假定 $P \neq NP$, 并且考虑人们如何可以证出它来. 可以试用两种一般方法: 对角线 / 归约法和布尔线路下界方法.

在可计算性理论中起始于停机问题的对角线和归约方法, 被成功地用来证明了许多不可判定问题. 它也成功地用于复杂性理论中对许多非常难的判定问题的超指数下界的证明. 例如只有加法的 Presburger 算术是一种可判定理论, Fischer 和 Rabin [FR74] 证明了用任何 Turing 机判定这一理论在最坏情况下要用至少 $2^{2^c n}$ 步, 其中 $c > 0$. 一般地使用对角线方法和归约方法所得之下界可以相对化; 即他们可继续把这方法用于问题本身和求解问题的 Turing 机可对一任意的信息源集 A 提出元素归属问题的环境方面, 但在 [BGS75] 中证明了, 存在信息源 A , 相对于它 $P=NP$, 这就启示了对角线 / 归约方法不能被用来区分这两个类. 有趣的是相对于 脱殊 (generic) 的信息源, $P \neq NP$ [BI87, CIY97].

Bool 线路是一有穷的无圈图, 其中每个非输入的结点, 或称为门, 都用一 Bool 联接词作为标记; 典型的 Bool 联接词有 {与, 或, 非 (AND, OR, NOT)}. 输入结点则标以变元 $x_1, \dots, x_n$, 且对每个变元赋以值 0 或 1. 在每个门处, 包括输出门, 线路按显然的方式计算出一个位值. 不难看出如果 L 是一 $\{0, 1\}$ 上的语言且是在 P 中的, 那么有一多项式大小的 Bool 线路族 $\{B_n\}$ 使得 B_n 有 n 个输入, 且对每个长度为 n 的位串 w , —— w 施于 B_n 的 n 个输入结点, 则 B_n 的输出位是 1 当且仅当 $w \in L$. 此时我们称 $\{B_n\}$ 计算 L .

因此要证出 $P \neq NP$ 只要证明解出某特定 NP 完全的问题 (如 3-SAT) 的 Bool 线路的任意族的大小有超多项式的大小. 追溯到 1949 年, Shannon [Sha49] 证明了对几乎所有 Bool 函数 $f: \{0, 1\}^n \rightarrow \{0, 1\}$, 任何计算 f 的 Bool 线路至少要 $2^n/n$ 个门. 但是对 “明晰

地给定”的 Bool 函数要找出超线性下界的一切企图都失败了；迄今为止被证明的最好的下界是 $4n$. Razborov 和 Rudich[RR97] 藉指出迄今所用的一切方法可划归为“自然证明”从而解释了失败的原因，当假定了一定的复杂性理论的猜测下，对一般线路下界的自然证明注定是要失败的。

在一个一般的计算模型上来证明有趣的下界方面，复杂性理论面临的失败要比在证明 $P \neq NP$ 的问题上所面临的失败更为普遍。和目前知识相协调的是，不但可满足性可有一多项式时间算法，而且在多带 Turing 机上它还可以有线性时间的算法。对 Karp 原来的文章 [Kar72] 中的所有 21 个问题可以得到相同的结果。我们知道有不同的复杂性类，但不能加以证明。例如考虑复杂性类的包含关系的如下序列

$$\text{LOGSPACE} \subseteq P \subseteq NP \subseteq \text{PSPACE}.$$

用一简单的对角线方法可以证明第一个类是最后一个类的真子类，但是我们无法证明任何相邻的包含关系是真包含。

作为另一个例子，令线性大小类 (LINEAR-SIZE) 是可用一大小为 $O(n)$ 的 Bool 线路的族 $\{B_n\}$ 计算的 $\{0,1\}$ 上的语言类。尚不知 P 或者 NP 是线性大小类的一子集，尽管 Kannan[Kan82] 证明了存在多项式分层 (NP 的一种推广) 中的语言不在线性大小类之内。因为若 $P=NP$ ，则多项式分层就坍塌到 P ，于是我们有

命题 2 若 P 是线性大小类的子集，则 $P \neq NP$

这一命题可以解释为证明 $P \neq NP$ 的一种方法，但是更普遍的信念是命题的假设不成立。

复杂性理论中的一个基本问题是当人们愿意接受一小概率的出错可能性时，是否一随机位数的资源可以用来大大地加快某种语言的识别。BPP 类是由一随机多项式时间算法识别、且对每个输入其出错率最多是一指数小的出错概率的一切语言 L 所组成的类。当然 $P \subseteq BPP$ ，但不知道这包含关系是否是真包含。素数集是在 BPP 之内 [SS77]，但是不知道它是否在 P 之中。认为 $BPP=P$ 的理由是，在使用一确定性拟随机数产生器当作“随机的”数位资源时，随机算法常常取得成功。

在 $P=BPP$ 问题和 $P=NP$ 问题之间有一种间接的但是很有趣的联系。令 E 是可在指数时间识别的语言类；即它是对某 $c > 0$ 满足 $T_M(n) = O(2^{cn})$ 的某 Turing 机 M 使得 $L = L(M)$ 的语言 L 组成的类。令 A 是在 E 中需要指数线路复杂性的某语言，于是有

断言 A: 有 $L \in E$ 和 $\epsilon > 0$ ，使得对每个计算 L 的线路族 $\{B_n\}$ 和对一切充分大的 n ， B_n 至少有 $2^{\epsilon n}$ 个门。

命题 3 若 A 成立则 $BPP=P$ 。若非 A 成立则 $P \neq NP$ 。

第一个蕴涵式是 Impagliazzo 和 Wigderson[IW97] 证明的一个漂亮的定理；第二个是由 V.Kabanets 所做的有趣的观察得来的，它是命题 2 的加强命题。实际上 Kabanets 由一个比非 A 更弱的假设推出 $P \neq NP$ ：即每个 L 中的语言可被一 Bool 线路族 $\{B_n\}$ 计算，使得对无穷多个 n ， B_n 比计算任意 Bool 函数 $f: \{0,1\}^n \rightarrow \{0,1\}$ 所需的最大数要少的门。但对此假设是否成立没有公认一致的看法。

附录： Turing 机的定义

Turing 机 M 由具有在一无穷带上读 / 写的头的有穷状态控制 (即一有穷程序) 组成. 这带子被分成方格, 每个方格可以存贮包括空白符号 b 在内的一有穷字母表 Γ 中一个符号. 每个机器 M 有一特定的输入字母表 Σ , Σ 是 Γ 的不含空白符号 b 在内的一子集. 在一个计算的每一步, M 处于一特定的有穷的可能的状态集 Q 中的某个状态 q . 开始时 Σ 上的一有穷输入串被写在带子上的相邻的方格上, 所有其他方格是空白的 (含 b), 读头注视输入串的最左符号并且 M 处于初始状态 q_0 . 在每一步, M 处于一状态 q 并且读头注视一含有某个带符号 s 的带子上的方格, 所执行的动作依赖于 (q, s) 而且被机器的转换函数 (或程序) δ 所指定, 动作包括在被注视的方格内打印一符号, 把读写头左移或右移一格, 并且进入一个新状态.

严格地说, Turing 机 M 是一数组 $\langle \Sigma, \Gamma, Q, \delta \rangle$, 其中 Σ, Γ, Q 是满足 $\Sigma \subseteq \Gamma$ 和 $b \in \Gamma - \Sigma$ 的有穷非空集. 状态集含三个特定状态 $q_0, q_{accept}, q_{reject}$ (初始状态, 接受状态, 拒绝状态). 转换函数 δ 满足

$$\delta : (Q - \{q_{accept}, q_{reject}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{-1, 1\}.$$

若 $\delta(q, s) = (q', s', h)$, 则其解释是若 M 处于状态 q 正注视符号 s , 那么 q' 是新的状态, s' 是被打印的符号, 根据 h 取值为 -1 或 1 读写头在带上向左 (或向右) 移一格.

我们假设集 Q 和 Γ 互不相交.

M 的一配置是形如 xqy 的串, 其中 $x, y \in \Gamma^*$, y 不是空串且 $q \in Q$.

配置 xqy 的含义为 M 处于状态 q , xy 在其带上, 读写头注视 y 的最左符号.

若 C 和 C' 是配置, 称 $C \xrightarrow{M} C'$ 若 $C = xqsy, \delta(q, s) = (q', s', h)$ 且如下条件之一成立:

$C' = xs'q'y$ 且 $h = 1$ 且 y 非空串.

$C' = xs'q'y$ 且 $h = 1$ 且 y 是空串.

$C' = x'q'as'y$ 且 $h = -1$ 且对 $a \in \Gamma, x = x'a$

$C' = q'bs'y$ 且 $h = -1$ 且 x 为空串.

称一个配置 xqy 是停机配置, 若 $q \in \{q_{accept}, q_{reject}\}$. 注意对每个非停机配置 C 有唯一的配置 C' 使得 $C \xrightarrow{M} C'$.

M 对输入 $w \in \Sigma^*$ 的计算是唯一的配置序列 $C_0, C_1, \dots$ 使得 $C_0 = q_0w$ (或当 w 为空串时 $C_0 = q_0b$) 且对计算中每个 C_{i+1} 的 i 有 $C_i \xrightarrow{M} C_{i+1}$, 且要么序列是无穷序列, 要么序列停止于一停机配置. 如果计算是有穷的, 那么其步数是配置的数目减一; 否则步数是无穷的. 我们称 M 接受 w , 当且仅当计算是有穷的且最后的配置包含状态 q_{accept} .

参考文献

- [ARS78] L. Adleman, R. L. Rivest, and A. Shamir. A method for obtaining digital signature and public-key cryptosystems. *Communications of the ACM*, 21(2), 1978.
- [BCSS98] Lenore Blum, Felipe Cucker, Michael Shub, and Steve Smale. *Complexity and Real Computation*. Springer-Verlag, New York, 1998.
- [BGS75] Baker, Gill, and Solovay. Relativizations of the $P=?NP$ question. *SICOMP: SIAM Journal on Computing*, 1975.
- [BI87] M. Blum and R. Impagliazzo. Generic oracles and oracle classes. In Ashok K. Chandra, editor, *Proceeding of the 28th Annual Symposium on Foundations of Computer Science*, pages 118–126, Los Angeles, CA, October 1987. IEEE Computer Society Press.
- [Cob64] A. Cobham. The intrinsic computational difficulty of functions. In Yehoshua Bar-Hillel, editor, *Proceedings of the 1964 International Congress for Logic, Methodology, and Philosophy of Science*, pages 24–30. Elsevier/North-Holland, 1964.
- [Coo71] Stephen A. Cook. The complexity of theorem-proving procedures. In *Conference Record of Third Annual ACM Symposium on Theory of Computing*, pages 151–158, Shaker Heights, Ohio, 3–5, 1971 1971.
- [Coo90] Stephen A. Cook. Computational complexity of higher type functions. In Ichiro Statake, editor, *Proceedings of the International Congress of Mathematicians, Kyoto, Japan, 1990*, pages 55–69. The Mathematical Society of Japan, Springer-Verlag, 1991.
- [CIY97] Stephen Cook, Russell Impagliazzo, and Tomoyuki Yamakami. A tight relationship between generic oracles and type-2 complexity theory. *Information and Computation*, 137(2): 159–170, 15 September 1997.
- [CLR90] Thomas Cormen, Charles Leiserson, and Ronald Rivest. *Introduction to Algorithms*. McGraw Hill, New York, 1990.
- [Edm65] Jack Edmonds. Minimum partition of a matroid into independent subsets. *J. Res. Nat. Bur. Standards Sect. B*, 69: 67–72, 1965.
- [FR74] M. J. Fischer and M. O. Rabin. Super-exponential complexity of Presburger arithmetic. In R. M. Karp, editor, *Complexity of Computation*, volume 7, pages 27–41. American Mathematical Society, Providence RI, 1974.
- [CJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability, a Guide to the Theory of NP-Completeness*. W. H. Freeman and Co., San Francisco, 1979.
- [Gol99] Oded Goldreich. *Modern Cryptography, Probabilistic Proofs and Pseudorandomness*. Springer-Verlag, Berlin Germany, 1999. ISBN 3-540-64766-X.
- [IW97] Impagliazzo and Wigderson. $P=BPP$ if E requires exponential circuits: Deran-

- domizing the XOR lemma. In *ACM Symposium on Theory of Computing (STOC)*, 1997.
- [Kan82] Ravi Kannan. Circuit-size lower bounds and non-reducibility to sparse sets. *Information and Control*, 55: 40–56, 1982.
- [Kar72] Richard M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103, New York, 1972. Plenum Press.
- [Lev73] L. Levin. Universal search problems (in Russian). *Problemy Peredachi Informatsii*, 9(3): 265–266, 1973. English translation in Trakhtenbrot, B. A.: A survey of Russian approaches to Perebor (brute-force search) algorithms. *Annals of the History of Computing*, 6 (1984), pages 384–400.
- [Mil76] G. L. Miller. Reimann's hypothesis and tests for primality. *J. Comput. System Sci*, 13: 300–317, 1976.
- [Pra75] V. R. Parth. Every prime has a succinct certificate. *SIAM Journal on Computing*, 4(3): 214–20, 1975.
- [RR97] Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55 (1): 24–35, August 1997.
- [RS83] N. Robertson and P. O. Seymour. Graph minors I–XIII. *Journal of Combinatorial Theory B*, 1983–1995.
- [Sha49] C. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28: 59–98, 1949.
- [Sho97] Peter Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal of Computing* 26: 1484–1509, 1997.
- [Sma98] Steve Smale. Mathematical problems for the next century. *MATHINT: The Mathematical Intelligencer*, 20, 1998.
- [SS77] R. Solovay and V. Strassen. A fast Monte-Carlo test for primality. *SIAM Journal on Computing*, 6(1): 84–85, March 1977.
- [Tur37] Alan Turing. On computable numbers with an application to the entscheidungs problem. *Proc. London Math. Soc.* ser. 2, 42: 230–265, 1936–7.

(杨东屏 译 冯琦 校)